COMSATS University Islamabad
Department of Computer Science

Course Information	
Course Code: **CSC354**	Course Title: **Machine Learning**
Credit Hours: **3(3,0)**	Lecture Hours/Week: **3**
Lab Hours/Week: **0**	Pre-Requisites: **None**

Text and Reference Books

Textbooks:

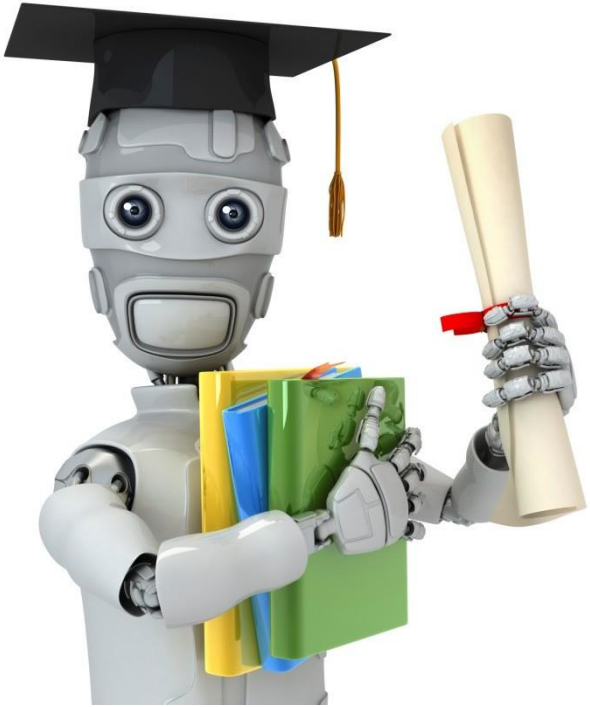
1. Introduction to Machine Learning, Ethem Alpaydin, MIT Press, 2010.
2. Machine Learning, Tom, M., McGraw Hill, 1997.

Reference Books:

1. Hands on Machine Learning with Scikit-Learn and TensorFlow, Aurelien Geron, O`Reilly Media, 2017.
2. Deep Learning with PyTorch – Essential Excerpts, Eli Stevens, Luca Antiga, Thomas Viehmann, Manning Publications, 2009.
3. Pattern Recognition and Machine Learning, Bishop, C., Springer-Verlag, 2007.

COMSATS University Islamabad
Department of Computer Science

- Zero tolerance for any misconduct and misbehavior
- Zero tolerance for cheating, if student found with any cheating material in quiz/exam will be marked zero
- Copied assignment will be marked zero
- Cross talk with class-fellows will result in strict disciplinary action
- Be in class on time, students who arrive after 5 min will be marked absent
- Submit assignments on portal on time, late assignment wouldn't be considered, and anything send via (WhatsApp etc) wouldn't be considered
- Emails or any message should be sent within working hours rest would not be entertained send email on (zeenatzulfiqar@cuiatd.edu.pk)



Machine Learning

Introduction

Supervised Learning

Supervised Learning

Supervised Learning (SL) is a type of **machine learning** where the model is trained on a **labeled dataset**.
“Labeled dataset” means every example in your training data has an **input** and a **correct output** (also called the target).
The goal is for the model to **learn the relationship between input and output** so that it can predict the output for new, unseen inputs.

How Supervised Learning Works

The process can be divided into steps:

Step 1: Collect Data

You need a dataset with features (inputs) and labels (outputs).

Example: Predicting house prices

Input features: size of the house, number of bedrooms, location

Output/Label: price of the house

Step 2: Split the Data

Usually, we split the dataset into:

Training set: Used to teach the model

Test set: Used to evaluate how well the model performs on unseen data

Example: 80% training, 20% testing

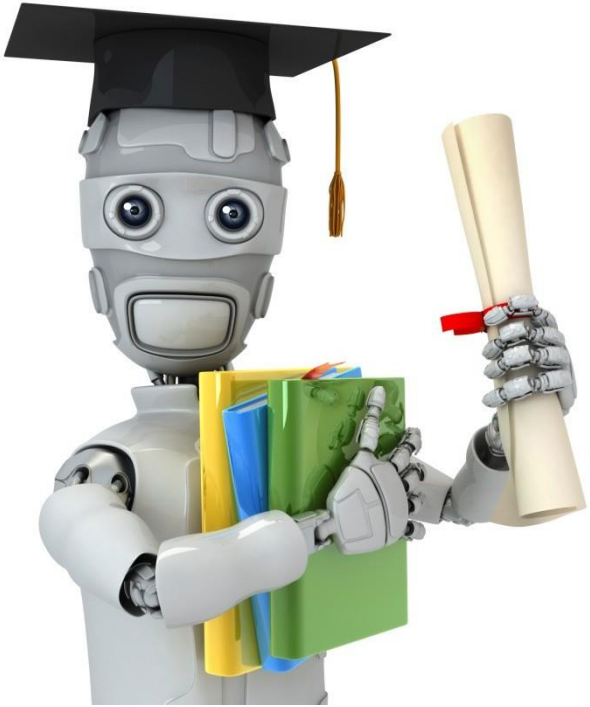
How Supervised Learning Works

Step 3: Choose a Model
Some common models for supervised learning:
Linear Regression → for predicting continuous values (like house prices)
Logistic Regression → for predicting probabilities / categories
Decision Trees → for classification or regression
Support Vector Machines (SVM) → for classification
Neural Networks → can do both classification and regression
Step 4: Train the Model
The model uses the training data to **learn the mapping between input and output**.
It adjusts its internal parameters to minimize the error between its predictions and the actual labels.
Example:
House with size 2000 sq ft → price 50,000$
Model predicts 48,000$ → error = 2,000$
Model adjusts parameters to reduce this error

How Supervised Learning Works

Step 5: Evaluate the Model
After training, test the model on the **test set**.
Common evaluation metrics:
Regression: Mean Squared Error (MSE), Root Mean Squared Error (RMSE)
Classification: Accuracy, Precision, Recall, F1-score

Step 6: Make Predictions
Once trained, you can feed **new input data** and the model will predict the output.
Example:
Input: house size 2500 sq ft, 3 bedrooms
Model predicts: price = 60,000$



Machine Learning

Introduction

Types of supervised Learning

Types of Supervised Learning

Supervised learning can be broadly divided into **two main types**:

1.Regression

2.Classification

Each type is used for a different kind of problem depending on the **type of output** we want the model to predict.

Types of Supervised Learning

1. Classification

Classification is a type of supervised learning where the model predicts **discrete labels** or **categories**.
Discrete means the output can only be one of a **limited number of classes**.
The goal is to **assign each input to the correct category**.
Examples:
Email Spam Detection
Input: Email text, sender, keywords
Output: Spam or Not Spam (2 classes → binary classification)
Tumor Detection in Medical Imaging
Input: MRI scan features
Output: Malignant or Benign (binary classification)
Digit Recognition (0-9)
Input: Image of a handwritten digit
Output: Predicted digit (10 classes → multi-class classification)
Sentiment Analysis
Input: Customer review text
Output: Positive, Negative, or Neutral (3 classes → multi-class classification)

Types of Supervised Learning

Regression

is a type of supervised learning where the model predicts a **continuous numeric value**.
Continuous means the output can take **any real number** within a range, not just categories.
The goal is to **find the relationship between input features and a numeric output**.

Examples:

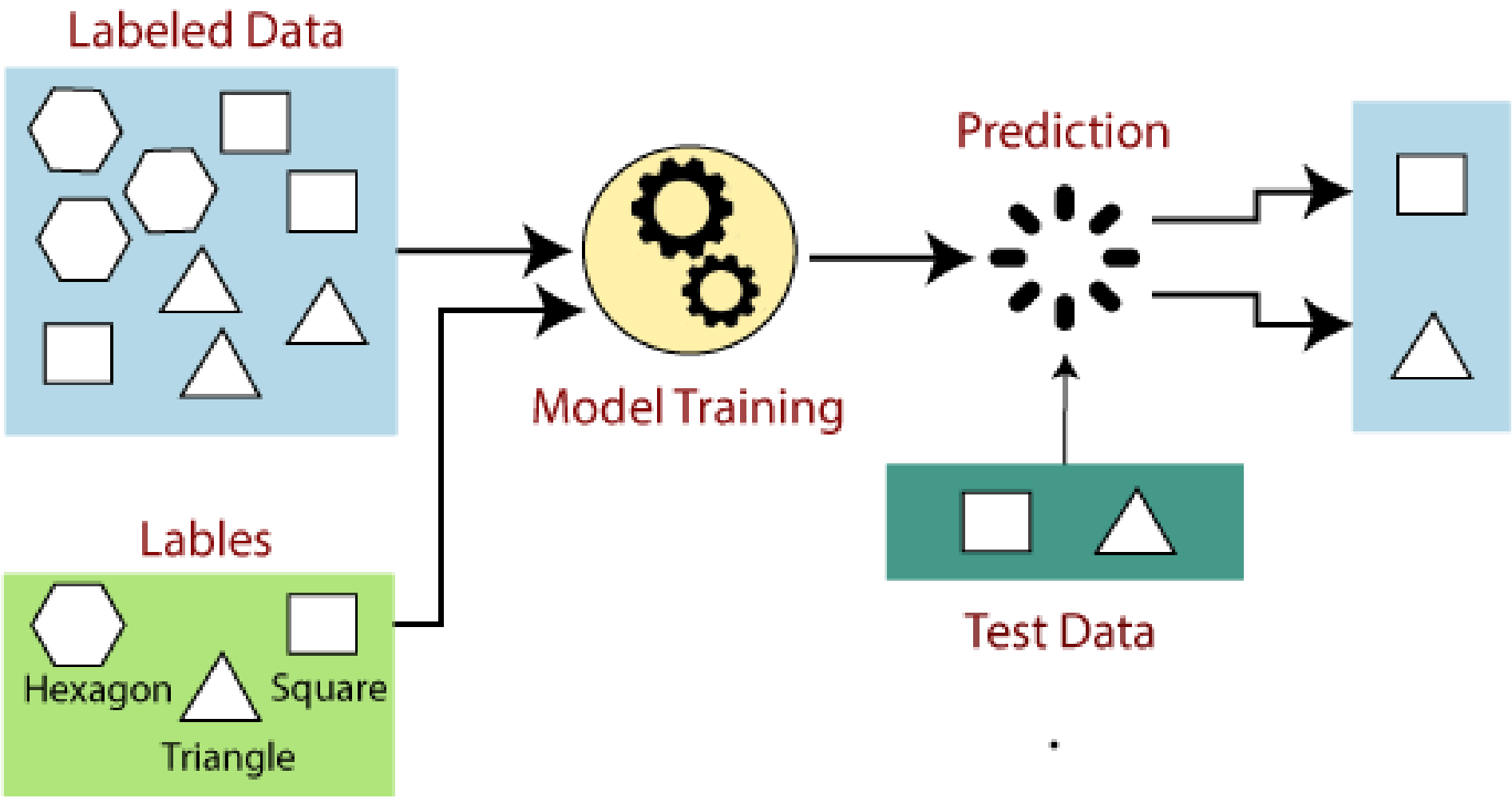
House Price Prediction

Input: Size of house, number of bedrooms, location
Output: Price in dollars
Example function:

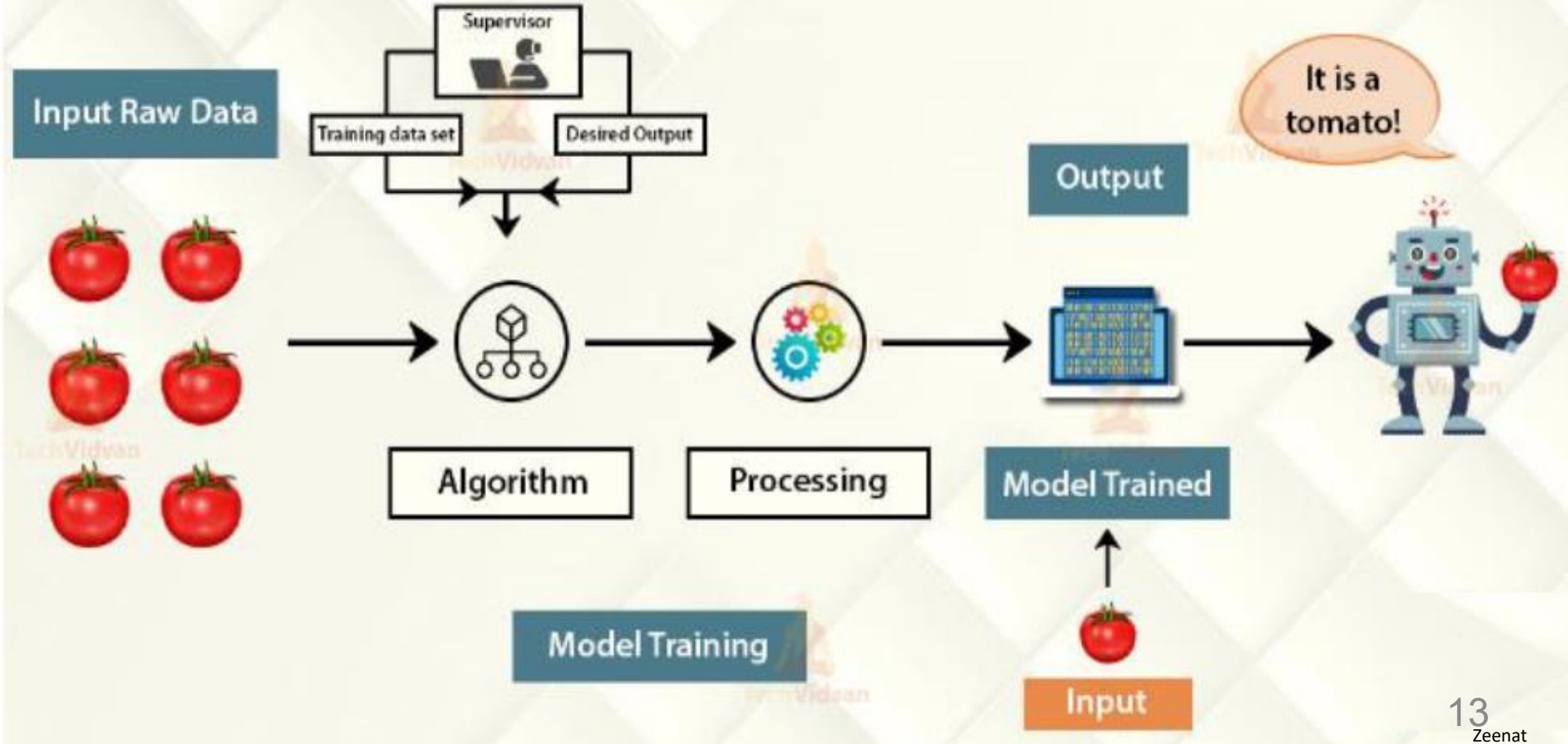
$$\text{Price} = 50{,}000 + 100 \times \textit{Size (sq ft)}$$

Stock Price Prediction

Input: Historical stock prices, market trends
Output: Predicted stock price


Labeled Data
Lables
Hexagon
Square
Triangle
Model Training
Prediction
Test Data

Supervised Learning in ML

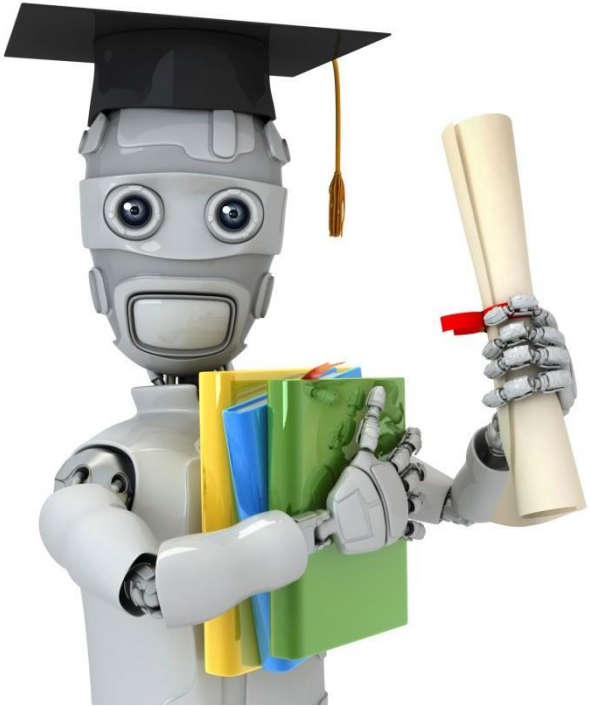


Zeenat

Identify whether the task is **Classification** or **Regression**

S.No.	Scenario / Problem Statement	Input Features	Output (Target)	Type
1	Predicting House Prices	House size (sq ft), number of bedrooms, location	Price in USD	?
2	Email Spam Detection	Email text, sender, keywords	Spam / Not Spam	?
3	Weather Forecasting	Day of the year, humidity, atmospheric pressure	Temperature (°C)	?
4	Student Exam Performance	Hours studied, attendance, previous marks	Pass / Fail	?
5	Stock Price Prediction	Historical stock prices, trading volume, market trends	Predicted stock price (USD)	?
6	Heart Disease Diagnosis	Age, blood pressure, cholesterol level, lifestyle factors	Disease / No Disease	?
7	Vehicle Fuel Efficiency Prediction	Engine size, vehicle weight, fuel type	Miles per gallon (MPG)	?
8	Sentiment Analysis of Customer Reviews	Text of customer review	Positive / Negative / Neutral	?
9	Employee Salary Prediction	Education level, years of experience, job title	Salary in USD	?
10	Handwritten Digit Recognition	Image pixels of handwritten digit	Digit label (0–9)	?

S.No.	Scenario / Problem Statement	Input Features	Output (Target)	Type
1	Predicting House Prices	House size (sq ft), number of bedrooms, location	Price in USD	Regression
2	Email Spam Detection	Email text, sender, keywords	Spam / Not Spam	Classification
3	Weather Forecasting	Day of the year, humidity, atmospheric pressure	Temperature (°C)	Regression
4	Student Exam Performance	Hours studied, attendance, previous marks	Pass / Fail	Classification
5	Stock Price Prediction	Historical stock prices, trading volume, market trends	Predicted stock price (USD)	Regression
6	Heart Disease Diagnosis	Age, blood pressure, cholesterol level, lifestyle factors	Disease / No Disease	Classification
7	Vehicle Fuel Efficiency Prediction	Engine size, vehicle weight, fuel type	Miles per gallon (MPG)	Regression
8	Sentiment Analysis of Customer Reviews	Text of customer review	Positive / Negative / Neutral	Classification
9	Employee Salary Prediction	Education level, years of experience, job title	Salary in USD	Regression
10	Handwritten Digit Recognition	Image pixels of handwritten digit	Digit label (0–9)	Classification



Machine Learning

Supervised learning Techniques

K-Nearest Neighbors (KNN)

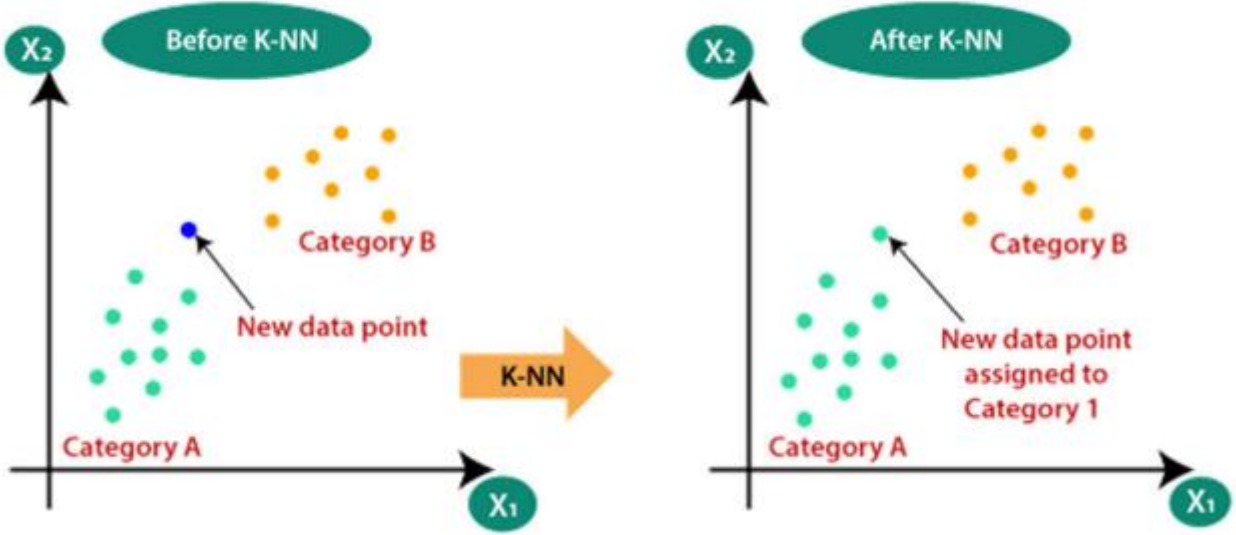
- KNN is a **supervised learning algorithm** used for **classification and Regression** tasks.
- KNN is a **lazy learner:** meaning it **doesn't build a mathematical model.** Instead, it **memorizes the training data and makes predictions based on similarity.**
- The **main idea** behind KNN is to **find the k-nearest data points** to a **given test data point** and use these nearest neighbors to make a prediction. The value of **k** is a **hyperparameter** that needs to be tuned, and it represents the number of neighbors to consider.

K-Nearest Neighbors (KNN)

- It works on the idea that **similar data points are located close to each other** in feature space. When a new input is given, KNN finds the **K closest data points (neighbors)** using a **distance measure such as Euclidean distance.**
- **In classification,** it **assigns the class based** on **majority voting** among neighbors, while **in regression**, it predicts the **average value**.
- In **KNN regression**, the output is a **number (continuous value)**, not a class.
- So instead of voting:

The algorithm:

- Finds **K nearest neighbors**
- Takes their **output values**
- Computes the **average (mean)**
- Uses that as the prediction


Before K-NN
X2
Category B
New data point
Category A
X1
K-NN
After K-NN
X2
Category B
New data point
assigned to
Category 1
Category A
X1

Working of KNN (Binary Case)

Step 1: Choose K: Select number of neighbors (e.g., K = 3 or 5,7)
Step 2: Calculate Distance: Compute distance between:
•New point
•All training data
(usually Euclidean distance)
Step 3: Find K Nearest Neighbors: Pick the **closest K points**
Step 4: Count Classes (Voting): Count how many belong to each class:
•**Class 0**
•**Class 1**
Step 5: Assign Class: Choose the class with **maximum votes**

Multi-class Classification in KNN

Instead of 2 classes, we have **more than 2 classes**

Examples:

- Digits: 0–9
- Fruits: Apple, Banana, Orange
- Attack types: DoS, Probe, Normal

KNN will choose **one class from many classes**

Working of KNN (Multi-class)

- First, **a new data point** is given whose class is unknown.
- Then, **choose the value of K** (number of nearest neighbors).
- Next, **calculate the distance between the new data point and all training data points.**
- After that, sort all the distances from smallest to largest.
- Then, select the K nearest neighbors based on the smallest distances.
- Now, check the class labels of these selected neighbors.
- Count how many neighbors belong to each class.
- **Then, apply majority voting to find the class with the highest count**.
- Finally, assign that class as the prediction for the new data point.

Problems in KNN Classification

High Prediction Time in KNN:

KNN stores all training data and does **not train a model in advance**. When a new data point comes**, it must calculate the distance to every training point. For large datasets, this means thousands or millions of distance calculations.** After that, it needs to **sort distances** to find the K nearest neighbors and then **vote** to assign the class. This process happens **every time a prediction is made**, making KNN **slow for large datasets**

Solutions for High Prediction Time:

1. **Use smaller training set**
    1. Reduce the number of points using **sampling** or **prototype selection**.
2. **Use efficient data structures**
    1. **KD-Tree** or **Ball-Tree** can search nearest neighbors **faster**.
3. **Use Approximate Nearest Neighbor (ANN)**
    1. Don’t check every point exactly; use **approximate methods** for speed.
4. **Dimensionality reduction**
    1. Reduce the number of features using **PCA** or **feature selection** → less data to compare → faster predictions.

Sensitive to Value of K in KNN:

The value of **K** in KNN decides how many neighbors the algorithm looks at to make a prediction. If **K is too small**, the algorithm may follow noisy or outlier points, causing **overfitting**. On the other hand, if **K is too large**, it considers too many points, including distant ones from other classes, which can smooth out important patterns and cause **underfitting**. Therefore, choosing the **right K** is very important to get accurate predictions.

Tie Problem (Voting Issue)

In KNN classification, the prediction is made by **majority voting** among the K nearest neighbors. Sometimes, there is a **tie**, meaning two or more classes have the same number of votes. For example, if Class A = 2 and Class B = 2, the model cannot decide which class to choose. This creates a **confusion** during prediction. To avoid ties, it is better to choose **odd values of K**. Another **solution is distance-weighted voting,** where closer neighbors have more influence.

Solution 1: Choose odd K

- If **K is odd**, you **cannot have a tie** (for 2 classes).
- Example: K = 5
  - 3 neighbors vote for A, 2 for B → A wins.

Tip: Always pick **odd K for 2 classes** to reduce ties.

Solution 2

Measure distance of each neighbor from the point you want to classify.
Assign a weight to each neighbor:
Closer → bigger weight
Farther → smaller weight
Add up the weighted votes for each class.
The class with the highest total weight wins.

Sensitive to Noise in KNN:

- KNN decides the class of a new point based on its **nearest neighbors**.
- **Noise** means a data point is **wrongly labeled** in the training set.
- **Example: Suppose a new point is near 3 neighbors:**

Normally, the **new point should be Red,** but because of the noisy Blue neighbor, KNN may **predict Red or Blue incorrectly**.

So, even one wrong neighbor can affect the prediction.

Neighbor	Class
1	Red
2	Red
3	Blue (wrong label, noisy)

Solutions to Reduce Noise Problem in KNN:

1. **Use Larger K:**
    1. Increasing K reduces the effect of a single noisy point because **majority voting is more stable**.
    2. Example: K = 5 or 7 instead of 1 or 3.
2. **Weighted Voting:**
    1. Give **closer neighbors more weight** in voting.
    2. Noisy neighbors that are farther away will have **less influence**.
3. **Data Cleaning / Noise Removal:**
    1. Detect and **remove mislabeled or outlier points** from the training dataset before using KNN.
4. **Distance-Based Filtering:**
    1. Ignore neighbors that are **too far** or seem inconsistent with the majority class.

Feature Scaling Problem in KNN:

KNN uses **distance** to find nearest neighbors. If one feature has very large values (like Salary) and another is small (like Age), the large feature will **dominate the distance**, and KNN may pick **wrong neighbors**. This can lead to **wrong predictions**. To fix this, we **scale the features,** so all values are in the same range, using methods like **Min-Max Scaling** or **Standardization**. After scaling, KNN can make **more accurate predictions**.

Working of KNN

- For two points **A(x_1, y_1)** and **B(x_2, y_2)**, the **Euclidean distance** is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Question:1

Suppose you have the following dataset with two features (X and Y) and corresponding labels: Now, consider a new data point with **x1 = 6.0** and **y1 = 7.0**.
Task: Using KNN with **k = 3**, predict the label for this new data point.

Data Point	X	Y	Label
1	2.0	3.0	A
2	3.0	4.0	A
3	5.0	6.0	B
4	7.0	8.0	B
5	10.0	10.0	A

Step 1: Write down the new point

New data point:

$$(x_1, y_1) = (6.0, 7.0)$$

We need to classify it using **k = 3 nearest neighbors**.

Step 2: Calculate distance from all points

We use **Euclidean distance formula**:

$$\text{Distance} = \sqrt{(x_1 - x_i)^2 + (y_1 - y_i)^2}$$

Compute for each data point:

1. **Point 1 (2,3):**

$$\sqrt{(6-2)^2 + (7-3)^2} = \sqrt{4^2 + 4^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66$$

2. **Point 2 (3,4):**

$$\sqrt{(6-3)^2 + (7-4)^2} = \sqrt{3^2 + 3^2} = \sqrt{9+9} = \sqrt{18} \approx 4.24$$

3. **Point 3 (5,6):**

$$\sqrt{(6-5)^2 + (7-6)^2} = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$$

4. **Point 4 (7,8):**

$$\sqrt{(6-7)^2 + (7-8)^2} = \sqrt{(-1)^2 + (-1)^2} = \sqrt{2} \approx 1.41$$

5. **Point 5 (10,10):**

$$\sqrt{(6-10)^2 + (7-10)^2} = \sqrt{(-4)^2 + (-3)^2} = \sqrt{16+9} = \sqrt{25} = 5.0$$

Step 3: Sort distances

Point	Distance	Label
3	1.41	B
4	1.41	B
2	4.24	A
5	5.0	A
1	5.66	A

Step 4: Pick k = 3 nearest neighbors

- The **3 closest points** are:
  - Point 3 → B
  - Point 4 → B
  - Point 2 → A

Step 5: Count votes

- Class A: 1 vote (Point 2)
- Class B: 2 votes (Points 3 & 4)

Step 6: Predict label

- Class with **most votes** = **B**

So, the new point **(6,7)** → **B**

Example:2

KNN

Classification (predicting classes)

e.g Predicting moving Genre

IMDb Rating	Duration	Genre
8.0 (mission impossible)	160	Action
6.2 (Gadar 2)	170	Action
7.2 (Rocky and Rani)	168	Comedy
8.2 (OMG2)	155	Comedy

Now predict the genre of "Barbie" move with IMDb rating 7.4 and duration 114

Step1: Calculation Distance (Euclidean distance)

Calculate the Euclidean distance b/w the new movie and each movie in the dataset

$$d(p,q) = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

$x_1 = 7.4$
$y_1 = 114$

Example:2

Distance to $(\overset{x_2}{8.0}, \overset{y_2}{160}) = \sqrt{((\overset{x_1}{7.4} - \overset{x_2}{8.0})^2 + (\overset{y_1}{114} - \overset{y_2}{160})^2)} = \approx 46.00$

Distance to $(6.2, 160) = \sqrt{((7.4 - 6.2)^2 + (114 - 170)^2)} = \approx 56.01$

Distance to $(7.2, 168) = \sqrt{((7.4 - 7.2)^2 + (114 - 168)^2)} = \approx 54.00$

Distance to $(8.2, 155) = \sqrt{((7.4 - 8.2)^2 + (114 - 155)^2} = \approx 41.00$

Step2:

Select k Nearest Neighbors

Suppose k=1 (Neighbors)

Small distance 41.00

Best value 5

k=3

Step3: Majority voting (A, C, C)
classification

Barbie move "C"

KNN Algorithm Example 1

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

KNN Algorithm Example 1

Sepal Length	Sepal Width	Species
5.2	3.1	?

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

KNN Algorith Example 1

- Step 1: Euclidean distance formulae.

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608

KNN Algorithm Example 1

- Step 1: Euclidean distance formulae.

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608
5.1	3.8	Setosa	0.707
7.2	3.0	Virginica	2.002
5.4	3.4	Setosa	0.36
5.1	3.3	Setosa	0.22
5.4	3.9	Setosa	0.82
7.4	2.8	Virginica	2.22
6.1	2.8	Verscicolor	0.94
7.3	2.9	Virginica	2.1
6.0	2.7	Verscicolor	0.89
5.8	2.8	Virginica	0.67
6.3	2.3	Verscicolor	1.36
5.1	2.5	Verscicolor	0.60
6.3	2.5	Verscicolor	1.25
5.5	2.4	Verscicolor	0.75

KNN Algorithm Example 1

- Step 2: Find the Rank

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.6080	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.6082	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

KNN Algorithm Example 1

- Step 2: Find the nearest neighbors, if k=2

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

If k = 1 – Setosa

If k = 2 – Setosa

KNN Algorithm Example 1

- Step 2: Find the nearest neighbors, if k=5

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

If k = 1 – Setosa

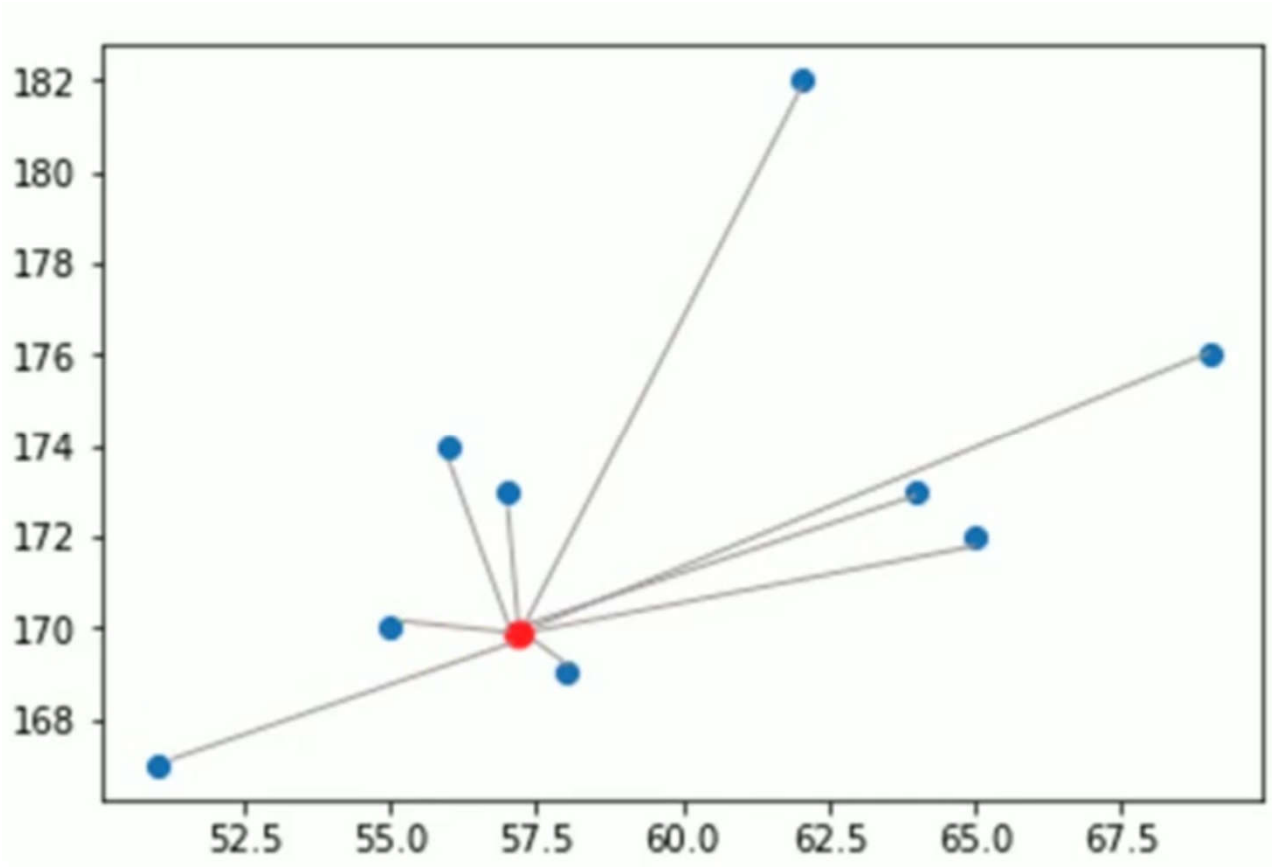
If k = 2 – Setosa

If k = 5 – Setosa

KNN Example 2:

Height (CM)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?

KNN Example 2:



THE DISTANCE FORMULA

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

KNN Example 2:

Height (CM)	Weight (KG)	Class	Distance
167	51	Underweight	6.7
182	62	Normal	13
176	69	Normal	13.4
173	64	Normal	7.6
172	65	Normal	8.2
174	56	Underweight	4.1
169	58	Normal	1.4
173	57	Normal	3
170	55	Normal	2
170	57	?	

THE DISTANCE FORMULA

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

KNN Example 2:

Height (CM)	Weight (KG)	Class	Distance	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9
170	57	?		

- If K=1, Normal
- If K=2, Normal
- If K=3, Normal
- If K=4, Normal

KNN Example 3

	Pepper	Ginger	Chilly	Liked
A	True	True	True	False
B	True	False	False	True
C	False	True	True	False
D	False	True	False	True
E	True	False	False	True

KNN Example 3

- The "Restaurant A" sells burger with optional flavors: Pepper, Ginger and Chilly.
- Every day this week you have tried a burger (A to E) and kept a record of which you liked.
- Using Hamming distance, show how the 3NN classifier with majority voting would classify

New Example - Q: pepper: false, ginger: true, chilly : true

- But How to calculate the distance for attributes with nominal or categorical values.
- Here we can use Hamming distance to find the distance between the categorical values.
- Let x1 and x2 are the attribute values of two instances.
- Then, in hamming distance, if the categorical values are same or matching that is x1 is same as x2 then distance is 0, otherwise 1.
- For example,
- If value of **x1 is blue** and **x2 is also blue** then the distance between x1 and x2 is **0**.
- If value of **x1 is blue** and **x2 is red** then the distance between x1 and x2 is **1**.

KNN Example 3

	Pepper	Ginger	Chilly	Liked	Distance
A	True	True	True	False	1 + 0 + 0 = 1
B	True	False	False	True	1 + 1 + 1 = 3
C	False	True	True	False	0 + 0 + 0 = 0
D	False	True	False	True	0 + 0 + 1 = 1
E	True	False	False	True	1 + 1 + 1 = 3

KNN Example 3

	Pepper	Ginger	Chilly	Liked	Distance	3NN
A	True	True	True	False	1 + 0 + 0 = 1	2
B	True	False	False	True	1 + 1 + 1 = 3	
C	False	True	True	False	0 + 0 + 0 = 0	1
D	False	True	False	True	0 + 0 + 1 = 1	2
E	True	False	False	True	1 + 1 + 1 = 3	

New Example - Q: pepper: false, ginger: true, chilly : true

Use Hamming Distance and

find the distance from Query Example (Q) to training examples (A-E)

KNN Example 3

New Example - Q: pepper: false, ginger: true, chilly : true

Use Hamming Distance and

find the distance from Query Example (Q) to training examples (A-E)

false

	Pepper	Ginger	Chilly	Liked	Distance	3NN
A	True	True	True	False	1 + 0 + 0 = 1	2
B	True	False	False	True	1 + 1 + 1 = 3	
C	False	True	True	False	0 + 0 + 0 = 0	1
D	False	True	False	True	0 + 0 + 1 = 1	2
E	True	False	False	True	1 + 1 + 1 = 3	